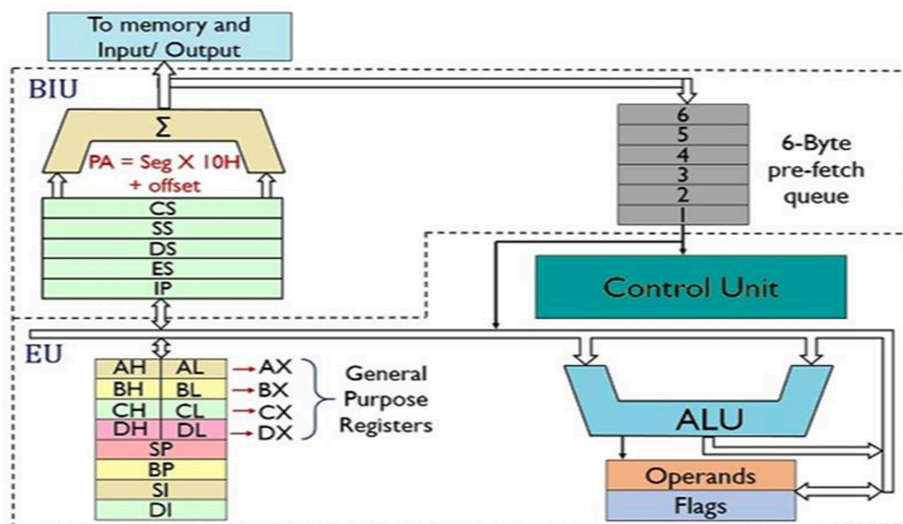


1.Explain the Architecture of 8086 Microprocessor with a Neat Block Diagram.

The Intel 8086 is an enhanced version of the 8085 microprocessor. It was designed by Intel in 1976. It is a 16-bit, N-channel HMOS microprocessor available in a 40-pin Dual Inline Package (DIP). It has 20 address lines and 16 data lines, allowing it to access 1 MB (2^{20} bytes) of memory and process 16-bit data in one operation. It provides 14 internal 16-bit registers for programming.

Architecture of 8086:



The architecture of the 8086 microprocessor is divided into two functional units:

1. **Bus Interface Unit (BIU)**
2. **Execution Unit (EU)**

This division enables pipelined operation, where the BIU fetches instructions while the EU executes the current instruction. This improves execution speed.

1. Bus Interface Unit (BIU)

The **Bus Interface Unit (BIU)** provides the interface between the microprocessor and external memory and I/O devices through the system bus.

Functions of BIU

- Generates the 20-bit physical address.
- Fetches instructions from memory.
- Transfers data between memory, I/O devices, and the processor.
- Maintains a 6-byte instruction prefetch queue.
- Supports pipelining by fetching instructions in advance.
- Performs memory read, memory write, I/O read, and I/O write operations.

Components of BIU

(a) Segment Registers

The BIU contains four 16-bit segment registers.

- CS (Code Segment): Stores the base address of the code segment.
 - DS (Data Segment): Stores the base address of the data segment.
 - SS (Stack Segment): Stores the base address of the stack segment.
 - ES (Extra Segment): Stores the base address of the extra segment.
-

(b) Instruction Pointer (IP)

- It is a 16-bit register.
 - It contains the offset address of the next instruction to be executed.
 - It works together with the Code Segment (CS) register.
-

(c) Address Generation Unit

The Address Generation Unit calculates the 20-bit physical address using the formula:

Physical Address = Segment Address × 10H + Offset Address

Example:

CS = 4042H

IP = 0580H

Physical Address = 40420H + 0580H = 409A0H (or as illustrated in the PPT example).

(d) 6-Byte Instruction Queue

The BIU contains a 6-byte FIFO instruction queue.

Functions:

- Stores prefetched instructions.
 - Allows simultaneous fetching and execution.
 - Fetches 2 instruction bytes in one memory cycle.
 - Refills the queue whenever at least two locations become vacant.
 - Improves processor performance through pipelining.
-

2. Execution Unit (EU)

The Execution Unit (EU) receives instructions from the instruction queue, decodes them, and executes them.

Functions of EU

- Decodes instructions.
 - Executes arithmetic and logical operations.
 - Controls program execution.
 - Uses registers to store operands and intermediate results.
 - Sends address requests to the BIU whenever memory access is required.
-

Components of EU

(a) Arithmetic Logic Unit (ALU)

The ALU performs:

- Addition
 - Subtraction
 - Multiplication
 - Division
 - Logical operations (AND, OR, XOR, NOT)
 - Comparison operations
-

(b) Control Unit

The Control Unit:

- Decodes instructions.
 - Generates control signals.
 - Coordinates all processor operations.
-

(c) General Purpose Registers

The 8086 has four 16-bit general-purpose registers, each of which can also be used as two 8-bit registers.

16-bit Register	8-bit Parts
AX	AH, AL
BX	BH, BL
CX	CH, CL
DX	DH, DL

(d) Pointer and Index Registers

- **SP (Stack Pointer):** Points to the top of the stack.
 - **BP (Base Pointer):** Accesses data in the stack segment.
 - **SI (Source Index):** Points to source data.
 - **DI (Destination Index):** Points to destination data.
-

(e) Flag Register

The 16-bit Flag Register stores the status of arithmetic and logical operations and controls processor execution.

Working of 8086

1. The BIU generates the physical address.
 2. The BIU fetches instructions from memory.
 3. Instructions are stored in the 6-byte instruction queue.
 4. The EU fetches instructions from the queue.
 5. The Control Unit decodes the instruction.
 6. The ALU executes the instruction.
 7. If memory access is needed, the EU requests the BIU.
 8. The BIU performs the required memory or I/O operation.
 9. While the EU executes the current instruction, the BIU fetches the next instruction, achieving pipelining.
-

Advantages of 8086 Architecture

- 16-bit data processing capability.
- Accesses up to 1 MB memory.
- Uses memory segmentation.
- Supports pipelining through the 6-byte instruction queue.
- Faster execution due to simultaneous instruction fetch and execution.
- Efficient communication with memory and I/O devices.

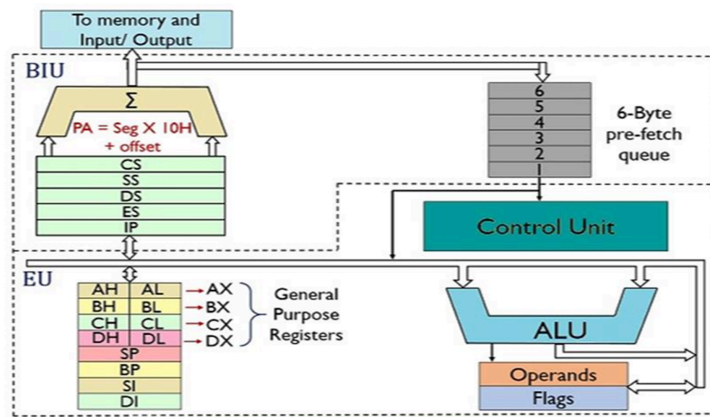
2.Explain the Internal Operation of 8086 Microprocessor with Neat Diagram

The internal operation of the 8086 microprocessor is based on the coordinated functioning of two independent units:

1. Bus Interface Unit (BIU)
2. Execution Unit (EU)

The BIU fetches instructions from memory while the EU decodes and executes them simultaneously. This simultaneous operation is called pipelining, which improves processor speed.

Block Diagram:



Internal Operation of 8086

The internal operation of the 8086 can be explained through the functioning of its two units.

1. Bus Interface Unit (BIU)

The Bus Interface Unit (BIU) provides communication between the processor and external memory and I/O devices through the system bus.

Functions of BIU

- Generates the 20-bit physical address.
- Fetches instructions from memory.
- Transfers data between memory and I/O devices.
- Maintains the 6-byte instruction prefetch queue.
- Performs memory read, memory write, I/O read, and I/O write operations.

Components of BIU

a) Segment Registers

The BIU contains four 16-bit segment registers.

- Code Segment (CS) – Holds the starting address of the program.
- Data Segment (DS) – Holds data, variables, and constants.
- Stack Segment (SS) – Holds stack data and return addresses.
- Extra Segment (ES) – Stores additional data, mainly for string operations.

b) Instruction Pointer (IP)

- It is a 16-bit register.
- Stores the offset address of the next instruction.
- Works together with the Code Segment (CS) register.

c) Address Generation Unit

The BIU contains an Address Generation Unit that produces the 20-bit physical address.

The physical address is calculated using the formula:

Physical Address = Segment Address × 10H + Offset Address

This enables the 8086 to access 1 MB of memory using 16-bit registers.

d) 6-Byte Instruction Queue

The BIU maintains a 6-byte instruction queue.

Features:

- Stores prefetched instructions.
 - Follows FIFO (First In First Out) principle.
 - Fetches 2 instruction bytes in one memory cycle.
 - Refills the queue whenever at least two bytes become empty.
 - Supports pipelined execution.
-

2. Execution Unit (EU)

The Execution Unit (EU) receives instructions from the queue, decodes them, and executes them.

Functions of EU

- Takes instructions from the queue.
 - Decodes instructions.
 - Performs arithmetic and logical operations.
 - Controls program execution.
 - Requests memory access from BIU whenever required.
-

Components of EU

a) Arithmetic Logic Unit (ALU)

The ALU performs:

- Addition
- Subtraction
- Multiplication
- Division
- AND
- OR
- XOR
- Comparison operations

b) Control Unit

The Control Unit:

- Decodes instructions.
 - Generates control signals.
 - Controls the execution of instructions.
 - Coordinates the operation of all internal units.
-

c) General Purpose Registers

The EU contains four 16-bit general-purpose registers.

- AX (Accumulator)
- BX (Base Register)
- CX (Count Register)
- DX (Data Register)

Each register can also be divided into two 8-bit registers:

- AH & AL
 - BH & BL
 - CH & CL
 - DH & DL
-

d) Pointer and Index Registers

The EU contains four pointer/index registers:

- SP (Stack Pointer) – Points to the top of the stack.
 - BP (Base Pointer) – Accesses data in the stack segment.
 - SI (Source Index) – Points to source data.
 - DI (Destination Index) – Points to destination data.
-

e) Flag Register

The 16-bit Flag Register stores the status of arithmetic and logical operations and controls processor execution.

Working of Internal Operation:

The internal operation of the 8086 takes place as follows:

Step 1-

The BIU generates the 20-bit physical address using the segment and offset addresses.

Step 2-

The BIU fetches instructions from memory.

Step 3-

The fetched instructions are stored in the 6-byte instruction queue.

Step 4-

The EU takes the instruction from the queue.

Step 5-

The Control Unit decodes the instruction.

Step 6-

The ALU executes arithmetic or logical operations.

Step 7-

If memory access is required, the EU sends a request to the BIU.

Step 8-

The BIU performs the required memory or I/O operation and transfers the data.

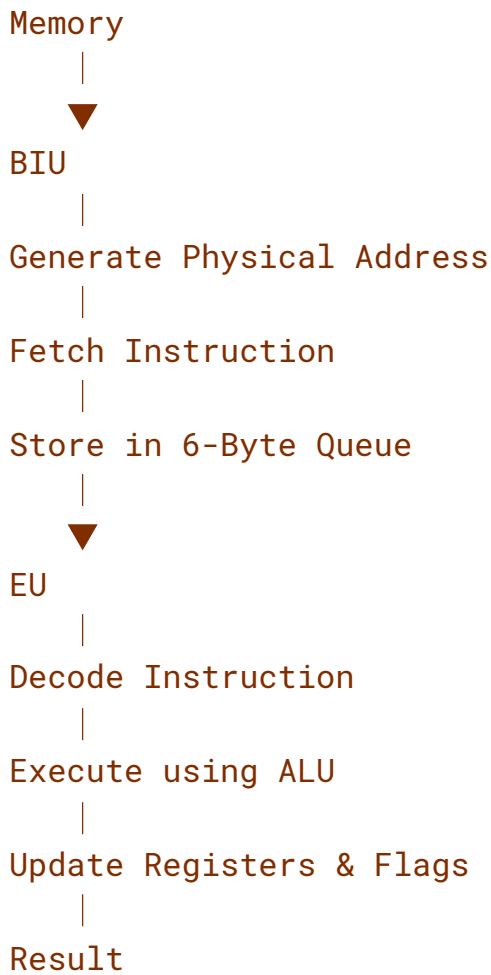
Step 9-

While the EU executes the current instruction, the BIU simultaneously fetches the next instruction. This overlapping of fetch and execution is called pipelining, which increases the execution speed of the processor.

Advantages of Internal Operation

- **Simultaneous instruction fetching and execution.**
 - **Faster execution through pipelining.**
 - **Efficient utilization of CPU resources.**
 - **Supports access to 1 MB memory.**
 - **Improves overall processor performance.**
-

Flow Diagram for Exam (Easy to Remember)



3.Explain the Addressing Modes of 8086 with Examples

Addressing mode is the method by which an instruction specifies the location of its operand (data). The operand may be present in a register, memory location, or within the instruction itself. The 8086 microprocessor supports 8 addressing modes.

Addressing Modes of 8086

The 8086 has 8 addressing modes, classified into:

A) Register and Immediate Addressing Modes (2)

1. Register Addressing
2. Immediate Addressing

B) Memory Addressing Modes (6)

3. Direct Addressing
3. Register Indirect Addressing
3. Based Addressing
3. Indexed Addressing
3. Based Indexed Addressing
3. Based Indexed with Displacement Addressing

1. Register Addressing Mode

In Register Addressing, the operand is stored in one of the 8-bit or 16-bit general-purpose registers.

Examples

```
MOV AX, CX
ADD AL, BL
```

Explanation

- Data is transferred directly between registers.
- No memory access is required.
- Execution is very fast.

2. Immediate Addressing Mode

In Immediate Addressing, the operand is given directly in the instruction.

Examples

```
MOV AL, 35H
MOV BX, 0301H
ADD AX, 4836H
```

Explanation

- The data is a constant.
- No memory location is used for the operand.

3. Direct Addressing Mode

In Direct Addressing, the offset address of the operand is specified directly in the instruction.

Example

```
ADD AL, [0301H]
```

Explanation

- Operand is stored in memory.
- Offset address is mentioned explicitly.

- Data is taken from the Data Segment (DS).
-

4. Register Indirect Addressing Mode

In Register Indirect Addressing, the offset address is stored in one of the registers:

- BX
- BP
- SI
- DI

Example

```
MOV AX, [BX]
```

Explanation

- The register contains the memory address.
 - The processor accesses memory using that register.
-

5. Based Addressing Mode

In Based Addressing, the effective address is obtained by adding a base register and a displacement.

Formula

$$\text{Effective Address} = \text{BX/BP} + \text{Displacement}$$

Examples

```
MOV AL, [BX+05]  
MOV AL, [BX+1346H]
```

Explanation

- BX is used for the Data Segment.
 - BP is used for the Stack Segment.
-

6. Indexed Addressing Mode

In Indexed Addressing, the effective address is obtained by adding an index register and a displacement.

Formula

Effective Address = SI/DI + Displacement

Examples

```
MOV AX, [SI+05]  
MOV AX, [SI+1528H]
```

Explanation

- SI and DI are used as index registers.
 - Mainly used for arrays and string operations.
-

7. Based Indexed Addressing Mode

In this mode, the effective address is obtained by adding a base register and an index register.

Formula

Effective Address = BX/BP + SI/DI

Examples

```
ADD AX, [BX+SI]  
  
MOV CX, [BX+SI]
```

Explanation

- BX is used with the Data Segment.
 - BP is used with the Stack Segment.
-

8. Based Indexed with Displacement Addressing Mode

In this mode, the effective address is obtained by adding:

- Base Register
- Index Register
- Displacement

Formula

Effective Address = BX/BP + SI/DI + Displacement

Examples

```
MOV AX, [BX+SI+05]
```

```
MOV AX, [BX+SI+1235H]
```

Explanation

- This is the most flexible addressing mode.
 - Used for accessing complex data structures.
-

Addressing Modes for Control Transfer Instructions

1. Intersegment Direct

- Control transfers to another code segment.
- Both CS and IP are specified directly.

Example

```
JMP 5000H, 2000H
```

2. Intersegment Indirect

- Control transfers to another segment.
- The destination address is obtained indirectly from memory.

Example

```
JMP [2000H]
```

3. Intrasegment Direct

- Jump occurs within the same code segment.
- The displacement is given directly.

Example

```
JMP SHORT LABEL
```

4. Intrasegment Indirect

- Jump occurs within the same segment.

- The destination address is obtained from a register or memory location.

Example

JMP [BX]

Summary Table

Addressing Mode	Operand Location	Example
Register	Register	MOV AX, CX
Immediate	Instruction	MOV AL, 35H
Direct	Memory	ADD AL, [0301H]
Register Indirect	Memory through register	MOV AX, [BX]
Based	Base register + displacement	MOV AL, [BX+05]
Indexed	Index register + displacement	MOV AX, [SI+05]
Based Indexed	Base + Index	ADD AX, [BX+SI]
Based Indexed with Displacement	Base + Index + displacement	MOV AX, [BX+SI+05]

Advantages of Addressing Modes

- Provide flexibility in accessing data.
- Reduce program size.
- Support efficient array and string processing.
- Improve programming convenience.
- Enable access to registers, memory, and immediate data using different methods.

4.Explain the Instruction Set of 8086 with Examples

An Instruction Set is the complete collection of instructions that a microprocessor can execute. Each instruction tells the processor what operation to perform. The 8086 supports

a large number of instructions, which are classified into different groups based on their functions.

Classification of 8086 Instruction Set:

The instruction set of 8086 is classified into six groups:

1. Data Transfer Instructions
 2. Arithmetic Instructions
 3. Bit Manipulation (Logical) Instructions
 4. Program Execution Transfer Instructions
 5. String Instructions
 6. Processor Control Instructions
-

1. Data Transfer Instructions-

These instructions transfer data from one location to another. The source and destination may be registers, memory locations, or immediate data.

Common Instructions

- MOV
 - PUSH
 - POP
 - XCHG
 - IN
 - OUT
 - XLAT
-

(a) MOV Instruction

The MOV instruction copies a byte or word from the source operand to the destination operand.

Syntax

MOV Destination, Source

Examples

MOV CX, 045FH

MOV AX, DX

Note

- Source may be register, memory, or immediate data.
 - Destination may be register or memory.
 - Memory-to-memory transfer is not allowed.
-

(b) PUSH Instruction

The PUSH instruction places an operand onto the top of the stack.

Syntax

PUSH Operand

Example

PUSH BX

(c) POP Instruction

The POP instruction removes data from the top of the stack and stores it in the destination.

Syntax

POP Destination

Example

POP AX

Note: Destination can be a general-purpose register, a segment register (except CS), or a memory location.

(d) XCHG Instruction

The XCHG instruction exchanges the contents of the source and destination operands.

Syntax

XCHG Destination, Source

Example

XCHG DX, AX

Note

Two memory locations cannot be exchanged directly.

(e) IN Instruction

Transfers data from an input port to the accumulator.

Example

```
IN AX, 0028H
```

(f) OUT Instruction

Transfers data from the accumulator to an output port.

Example

```
OUT 0028H, AX
```

2. Arithmetic Instructions

These instructions perform arithmetic operations such as addition, subtraction, multiplication, and division.

Common Arithmetic Instructions

- ADD
 - ADC
 - SUB
 - SBB
 - INC
 - DEC
 - MUL
 - DIV
 - CMP
 - NEG
-

(a) ADD

Adds the source operand to the destination operand.

ADD AX, BX

(b) SUB

Subtracts the source operand from the destination operand.

SUB AX, BX

(c) INC

Increments the operand by 1.

INC AX

(d) DEC

Decrements the operand by 1.

DEC BX

(e) MUL

Multiplies unsigned numbers.

MUL BL

(f) DIV

Divides unsigned numbers.

DIV BL

(g) CMP

Compares two operands without storing the result.

CMP AX, BX

3. Bit Manipulation (Logical) Instructions

These instructions perform logical operations on binary data.

Common Instructions

- AND
 - OR
 - XOR
 - NOT
 - TEST
 - SHL
 - SHR
 - ROL
 - ROR
-

Examples

```
AND AX, BX
```

```
OR AX, BX
```

```
XOR AX, BX
```

```
NOT AX
```

Uses

- Bit masking
 - Bit setting
 - Bit clearing
 - Data manipulation
-

4. Program Execution Transfer Instructions

These instructions alter the normal sequence of program execution.

Common Instructions

- JMP
- CALL
- RET
- LOOP
- JZ
- JNZ
- JC

- JNC

Example

JMP NEXT

CALL DISPLAY

RET

LOOP AGAIN

These instructions are mainly used for branching, looping, and subroutines.

5. String Instructions

These instructions operate on strings or arrays of data.

Common Instructions

- MOVS
- CMPS
- SCAS
- LODS
- STOS

Uses

- Copying strings
 - Comparing strings
 - Scanning strings
 - Loading string data
 - Storing string data
-

6. Processor Control Instructions

These instructions control the operation of the processor.

Common Instructions

- NOP
- HLT
- WAIT
- ESC
- LOCK
- STI

- CLI

Functions

- Stop processor
- Enable/disable interrupts
- Synchronize processor operations

Summary Table

Instruction Group	Purpose	Examples
Data Transfer	Transfer data	MOV, PUSH, POP, XCHG
Arithmetic	Perform arithmetic	ADD, SUB, MUL, DIV
Bit Manipulation	Logical operations	AND, OR, XOR, NOT
Program Transfer	Change execution flow	JMP, CALL, RET, LOOP
String	Process strings	MOVS, CMPS, SCAS
Processor Control	Control processor	NOP, HLT, CLI, STI

Advantages of the 8086 Instruction Set

- Supports efficient data transfer.
- Performs arithmetic and logical operations quickly.
- Provides flexible program control through jumps and loops.
- Supports string manipulation.
- Includes processor control instructions for interrupt and execution management.

5.Explain Memory Segmentation in 8086 and Physical Address Calculation

Memory Segmentation is the process of dividing the available memory into smaller sections called segments. Each segment has its own base address, making memory access faster and more efficient.

The 8086 microprocessor has a 20-bit address bus, allowing it to access 1 MB (2²⁰ bytes) of memory. Since its registers are only 16-bit, memory is divided into segments to enable efficient addressing.

Need for Memory Segmentation

Memory segmentation is used for the following reasons:

- To increase the execution speed.
 - To improve the instruction fetching speed.
 - To enable the processor to access 1 MB memory using 16-bit registers.
 - To organize memory into logical sections.
 - To support simultaneous execution of different program parts.
-

Memory Segmentation in 8086

The 1 MB memory is divided into four segments, each having a maximum size of 64 KB.

The four memory segments are:

1. **Code Segment (CS)**
 2. **Data Segment (DS)**
 3. **Stack Segment (SS)**
 4. **Extra Segment (ES)**
-

Types of Memory Segments

1. Code Segment (CS)

Function

- Stores the program instructions (machine code).
- The Instruction Pointer (IP) provides the offset address.

Purpose

Used for instruction execution.

2. Data Segment (DS)

Function

- Stores data.
- Stores variables.
- Stores constants.

Purpose

Used for data processing operations.

3. Stack Segment (SS)

Function

- Stores the stack.
- Stores return addresses.
- Stores temporary data.
- Used during subroutine execution.

Associated Register

Stack Pointer (SP)

4. Extra Segment (ES)

Function

Stores additional data, mainly used by string instructions.

Offset Address

To access a particular memory location inside a segment, an offset address is required.

The offset address is 16 bits wide and is provided by the pointer or index registers.

Pointer Registers

- SP (Stack Pointer)
- BP (Base Pointer)

Index Registers

- SI (Source Index)
 - DI (Destination Index)
-

Physical Address Calculation

The 8086 generates a 20-bit physical address by combining:

- Segment Address
- Offset Address

Formula

Physical Address = Segment Address × 10H + Offset Address

Example (From PPT)

Given

- Code Segment (CS) = 4042H

- Instruction Pointer (IP) = 0580H

Step 1

Multiply the segment address by 10H.

$$4042H \times 10H = 40420H$$

Step 2

Add the offset.

$$\begin{array}{r} 40420H \\ +0580H \\ \hline 409A0H \end{array}$$

Physical Address

$$409A0H$$

This is the 20-bit physical address.

Advantages of Memory Segmentation

- Increases execution speed.
- Improves instruction fetching speed.
- Supports access to 1 MB memory.
- Organizes memory into logical sections.
- Enables efficient use of memory.
- Supports pipelining and multitasking.

Summary Table

Segment	Register	Purpose
Code Segment	CS	Stores program instructions
Data Segment	DS	Stores data, variables, constants
Stack Segment	SS	Stores stack and return addresses
Extra Segment	ES	Stores additional data for string operations

Important Registers Used

Register	Function
CS	Code Segment
DS	Data Segment
SS	Stack Segment
ES	Extra Segment
IP	Offset of next instruction
SP	Stack Pointer
BP	Base Pointer
SI	Source Index
DI	Destination Index

6.Explain General Purpose Registers and Segment Registers of 8086

The 8086 microprocessor contains 14 internal registers, each 16 bits (2 bytes) wide. These registers temporarily store data, addresses, instructions, and execution results, thereby increasing the execution speed of the processor.

Classification of Registers:

The registers in 8086 are classified into:

1. General Purpose Registers
2. Segment Registers
3. Pointer Registers
4. Index Registers
5. Instruction Pointer
6. Flag Register

1. General Purpose Registers

The 8086 has four 16-bit general-purpose registers.

- AX (Accumulator Register)
- BX (Base Register)
- CX (Count Register)
- DX (Data Register)

Each 16-bit register can be divided into two 8-bit registers.

16-bit Register	Higher Byte	Lower Byte
AX	AH	AL
BX	BH	BL
CX	CH	CL
DX	DH	DL

(a) Accumulator Register (AX)

Function

- Stores arithmetic and logical operation results.
- Used by multiplication and division instructions.
- Most frequently used register in ALU operations.

Example

```
MOV AX, 2500H
ADD AX, BX
```

(b) Base Register (BX)

Function

- Stores the base address of data.
- Used in memory addressing.
- Commonly used in based addressing mode.

Example

```
MOV AX, [BX]
```

(c) Count Register (CX)

Function

- Stores loop counters.
- Used by LOOP instruction.
- Used in shift and rotate operations.

Example

```
LOOP AGAIN
```

(d) Data Register (DX)

Function

- Stores data during multiplication and division.
- Used during input and output operations.

Example

```
MUL BX
```

2. Segment Registers

The 8086 contains four 16-bit segment registers.

They store the starting (base) addresses of memory segments.

The four segment registers are:

- Code Segment (CS)
 - Data Segment (DS)
 - Stack Segment (SS)
 - Extra Segment (ES)
-

(a) Code Segment Register (CS)

Function

- Holds the base address of the Code Segment.
 - Used together with the Instruction Pointer (IP).
 - Stores the program instructions.
-

(b) Data Segment Register (DS)

Function

- Holds the base address of the Data Segment.
 - Stores variables, constants, and data.
-

(c) Stack Segment Register (SS)

Function

- Holds the base address of the Stack Segment.
 - Used together with the Stack Pointer (SP).
 - Stores return addresses and temporary data.
-

(d) Extra Segment Register (ES)

Function

- Holds the base address of the Extra Segment.
 - Mainly used during string operations.
-

3. Pointer Registers

The 8086 has two pointer registers.

(a) Stack Pointer (SP)

Function

- 16-bit register.
- Points to the top of the stack.

- Used with the Stack Segment.
-

(b) Base Pointer (BP)

Function

- 16-bit register.
 - Points to data inside the stack segment.
 - Mainly used during subroutine execution.
-

4. Index Registers

There are two index registers.

(a) Source Index (SI)

Function

- Points to source data.
 - Used in string instructions.
-

(b) Destination Index (DI)

Function

- Points to destination data.
 - Used in string instructions.
-

5. Instruction Pointer (IP)

Function

- It is a 16-bit register.
 - Stores the offset address of the next instruction.
 - Works together with the Code Segment Register (CS).
-

6. Flag Register

Function

- It is a 16-bit register.
 - Stores the status of arithmetic and logical operations.
 - Controls processor execution.
 - Contains status flags and control flags.
-

Summary Table

Register	Function
AX	Arithmetic and logical operations
BX	Stores base address
CX	Loop counter
DX	Data, multiplication, division and I/O
CS	Stores code segment address
DS	Stores data segment address
SS	Stores stack segment address
ES	Stores extra segment address
SP	Points to top of stack
BP	Points to data in stack
SI	Points to source data
DI	Points to destination data
IP	Stores address of next instruction
Flag Register	Stores processor status and control information

Advantages of Registers

- Provide high-speed data storage.
- Reduce memory access time.
- Increase execution speed.
- Support arithmetic, logical, and memory operations.
- Improve processor performance.

7.Explain Instruction Queue and Pipelining in 8086

The Instruction Queue is a 6-byte FIFO (First In First Out) memory provided in the Bus Interface Unit (BIU) of the 8086 microprocessor. It stores instructions fetched from memory before they are executed.

The simultaneous fetching of instructions by the BIU and execution of instructions by the EU is called Pipelining. It improves the overall execution speed of the processor.

Instruction Queue

The Instruction Queue is located inside the Bus Interface Unit (BIU).

Features of Instruction Queue

- It has a capacity of 6 bytes.
 - It follows the FIFO (First In First Out) principle.
 - It stores instructions fetched from memory.
 - It supplies instructions to the Execution Unit (EU).
 - It supports pipelining by allowing simultaneous fetching and execution.
 - BIU fetches 2 instruction bytes in one memory cycle.
 - BIU starts refilling the queue when at least two locations become vacant.
-

Working of Instruction Queue

Step 1

The BIU fetches instructions from memory.

Step 2

The fetched instructions are stored in the 6-byte instruction queue.

Step 3

The Execution Unit (EU) takes the first instruction from the queue.

Step 4

The Control Unit decodes the instruction.

Step 5

The ALU executes the instruction.

Step 6

While the EU is executing the current instruction, the BIU fetches the next instructions and stores them in the queue.

This simultaneous operation continues until the program execution is completed.

Pipelining

Pipelining is a technique in which the Bus Interface Unit (BIU) fetches the next instruction while the Execution Unit (EU) executes the current instruction.

Thus, instruction fetching and instruction execution occur simultaneously, reducing processor idle time and increasing execution speed.

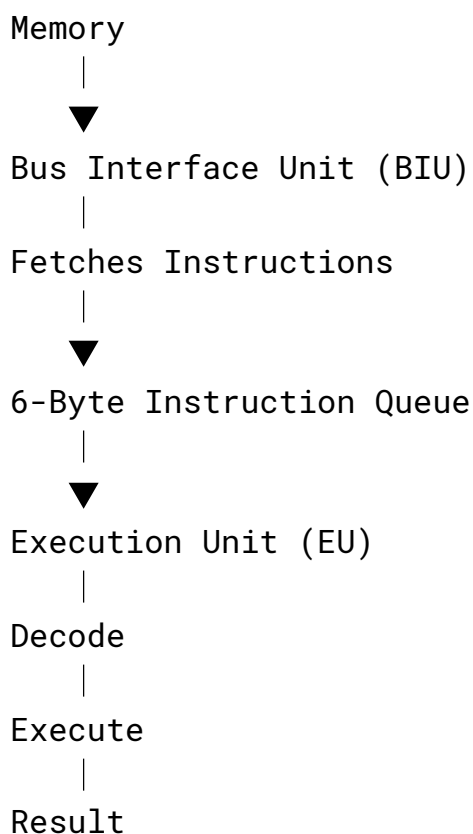
How Pipelining Works

The pipelining process takes place as follows:

1. BIU fetches the first instruction from memory.
2. The instruction is stored in the 6-byte instruction queue.
3. EU fetches the instruction from the queue and starts execution.
4. During execution, the BIU fetches the next instruction.
5. The next instruction is placed in the queue.
6. After completing the current instruction, the EU immediately executes the next instruction from the queue.

This overlapping of instruction fetch and instruction execution is known as pipelining.

Flow Diagram



Advantages of Instruction Queue

- Stores instructions before execution.
 - Reduces processor waiting time.
 - Ensures continuous instruction supply to the EU.
 - Supports pipelining.
 - Improves processor efficiency.
-

Advantages of Pipelining

- Increases execution speed.
 - Improves processor performance.
 - Reduces instruction execution time.
 - Allows simultaneous fetching and execution.
 - Maximizes CPU utilization.
 - Increases system throughput.
-

Difference Between Instruction Queue and Pipelining

Instruction Queue	Pipelining
Stores prefetched instructions	Executes and fetches instructions simultaneously
Located inside the BIU	Technique used by the processor
Capacity is 6 bytes	Improves processor speed
Works on FIFO principle	Uses BIU and EU together
Supplies instructions to the EU	Reduces execution time

8.Differentiate between Microprocessor and Microcontroller

Microprocessor

A microprocessor is a programmable electronic device fabricated on a single silicon chip. It consists of the Arithmetic Logic Unit (ALU), Control Unit, and Registers. It accepts binary data as input, processes it according to the instructions stored in memory, and produces the output. It requires external memory and I/O devices for operation.

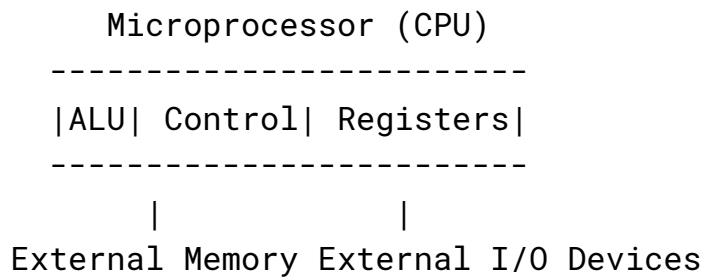
Microcontroller

A microcontroller is an electronic system that consists of a microprocessor, RAM, ROM, I/O ports, timers, and other peripheral devices on a single chip. It is a compact and low-cost device mainly used in embedded systems to control dedicated operations automatically.

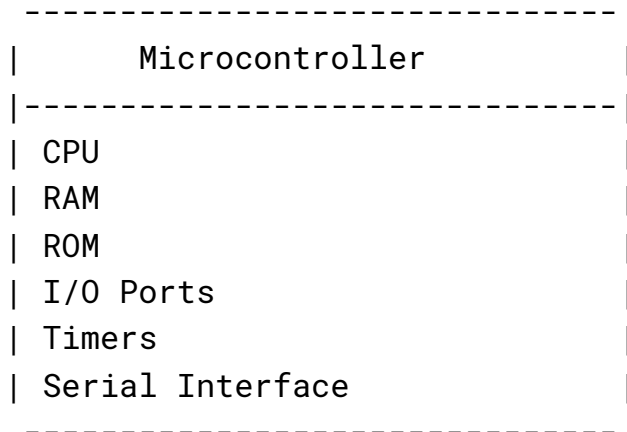
Difference Between Microprocessor and Microcontroller

Microprocessor	Microcontroller
Contains only CPU (ALU, Control Unit and Registers).	Contains CPU, RAM, ROM, I/O ports and peripherals on a single chip.
Requires external memory.	Memory is available internally.
Requires external I/O devices.	I/O ports are available internally.
Cost of the complete system is high.	Cost of the complete system is low.
Consumes more power.	Consumes less power.
Larger in size because of external components.	Compact in size.
Suitable for general-purpose computing.	Suitable for dedicated embedded applications.
Speed depends on external memory access.	Faster for dedicated control applications due to integrated peripherals.
Used in computers and laptops.	Used in washing machines, microwave ovens, refrigerators, air conditioners, etc.
More flexible for complex applications.	Optimized for a specific task.

Block Diagram of Microprocessor



Block Diagram of Microcontroller



Applications of Microprocessor

- Personal computers
- Laptops
- Workstations
- Servers
- Industrial computers

Applications of Microcontroller

According to the PPT, microcontrollers are widely used in embedded systems such as:

- Washing machines
- Air conditioners
- Refrigerators
- Microwave ovens
- Smart refrigerators
- Embedded systems
- IoT devices

Advantages of Microprocessor

- High processing capability.

- Suitable for general-purpose applications.
 - Can execute complex programs.
 - Easy to upgrade by adding external memory and peripherals.
-

Advantages of Microcontroller

- Compact design.
- Low cost.
- Low power consumption.
- High reliability.
- Ideal for embedded systems.
- Requires fewer external components.

9.Explain the Flag Register in 8086(psw)

The Flag Register is a 16-bit register in the Execution Unit (EU) of the 8086 microprocessor. It stores the status of arithmetic and logical operations performed by the ALU and also controls certain processor operations.

The Flag Register contains 9 active flags, which are divided into:

1. Status Flags
 2. Control Flags
-

Classification of Flags

The 8086 Flag Register consists of:

1. Status Flags (6)

- Carry Flag (CF)
 - Parity Flag (PF)
 - Auxiliary Carry Flag (AF)
 - Zero Flag (ZF)
 - Sign Flag (SF)
 - Overflow Flag (OF)
-

2. Control Flags (3)

- Trap Flag (TF)
 - Interrupt Flag (IF)
 - Direction Flag (DF)
-

1. Status Flags

Status flags indicate the result of arithmetic and logical operations.

(a) Carry Flag (CF)

- Set to 1 if an arithmetic operation generates a carry out of the most significant bit or a borrow during subtraction.
- Otherwise, it is cleared to 0.

Uses

- Multi-byte addition and subtraction.
- Unsigned arithmetic operations.

(b) Parity Flag (PF)

- Indicates the parity of the result.
- PF = 1 if the number of 1's in the result is even.
- PF = 0 if the number of 1's is odd.

(c) Auxiliary Carry Flag (AF)

- Set when a carry is generated from bit 3 to bit 4.
- Mainly used in BCD (Binary Coded Decimal) arithmetic operations.

(d) Zero Flag (ZF)

- Set to 1 if the result of an operation is zero.
- Cleared to 0 if the result is non-zero.

(e) Sign Flag (SF)

- Indicates the sign of the result.
- SF = 1 if the result is negative.
- SF = 0 if the result is positive.

The sign is determined by the Most Significant Bit (MSB).

(f) Overflow Flag (OF)

- Set when a signed arithmetic operation produces a result that exceeds the allowable range.
- Indicates arithmetic overflow.

2. Control Flags

Control flags control the execution of the processor.

(a) Trap Flag (TF)

- Enables single-step execution of a program.
 - Mainly used for debugging.
-

(b) Interrupt Flag (IF)

- Controls maskable interrupts.
 - IF = 1 → Interrupts are enabled.
 - IF = 0 → Interrupts are disabled.
-

(c) Direction Flag (DF)

- Controls the direction of string operations.
 - DF = 0 → String processing is performed in the forward direction.
 - DF = 1 → String processing is performed in the backward direction.
-

Summary Table

Flag	Name	Function
CF	Carry Flag	Indicates carry or borrow
PF	Parity Flag	Indicates even or odd parity
AF	Auxiliary Carry Flag	Used in BCD arithmetic
ZF	Zero Flag	Indicates zero result
SF	Sign Flag	Indicates positive or negative result
OF	Overflow Flag	Indicates signed overflow
TF	Trap Flag	Enables single-step execution
IF	Interrupt Flag	Enables/disables interrupts
DF	Direction Flag	Controls direction of string operations

Uses of Flag Register

- Indicates the result of arithmetic operations.
- Indicates the result of logical operations.
- Controls interrupt processing.
- Supports debugging through single-step execution.
- Controls string processing.
- Assists in decision-making during program execution.

Advantages of Flag Register

- Provides processor status information.
- Simplifies conditional branching.
- Supports arithmetic and logical operations.
- Helps in interrupt handling.
- Supports debugging.
- Controls string instructions.

10.Explain the Minimum Mode and Maximum Mode of 8086 with Differences

The 8086 microprocessor can operate in two modes depending on the value of the MN/M $\bar{X}$ (Minimum/Maximum) pin.

1. Minimum Mode
2. Maximum Mode

The operating mode is selected during system initialization.

1. Minimum Mode

The Minimum Mode is selected when the MN/M $\bar{X}$ pin is HIGH (1).

In this mode, the 8086 operates as a single processor, and all control signals required for memory and I/O operations are generated internally by the 8086.

Features of Minimum Mode

- Operates with only one microprocessor.
 - 8086 itself generates all control signals.
 - No external bus controller is required.
 - Simple hardware design.
 - Suitable for small computer systems.
 - Low cost and easy to implement.
-

Applications

- Personal computers
 - Small embedded systems
 - Stand-alone microprocessor systems
-

2. Maximum Mode

The Maximum Mode is selected when the MN/M $\bar{X}$ pin is LOW (0).

In this mode, the 8086 works with one or more processors (multiprocessor system). An external 8288 Bus Controller generates the required control signals.

Features of Maximum Mode

- Supports multiprocessor systems.
- Requires an external 8288 Bus Controller.
- Suitable for large computer systems.
- Allows coprocessors to work with the 8086.

- **More complex hardware design.**
 - **Higher performance for advanced applications.**
-

Applications

- **Multiprocessor systems**
 - **High-performance computer systems**
 - **Systems using coprocessors**
-

Difference Between Minimum Mode and Maximum Mode

Minimum Mode	Maximum Mode
Selected when $MN/M\bar{X} = 1$ (HIGH)	Selected when $MN/M\bar{X} = 0$ (LOW)
Single processor system	Multiprocessor system
Control signals generated by 8086	Control signals generated by 8288 Bus Controller
External bus controller not required	External bus controller required
Hardware is simple	Hardware is complex
Lower cost	Higher cost
Suitable for small systems	Suitable for large systems
Does not support coprocessors	Supports coprocessors

Advantages of Minimum Mode

- **Simple circuit design.**
 - **Low hardware cost.**
 - **Easy to implement.**
 - **Suitable for stand-alone systems.**
-

Advantages of Maximum Mode

- **Supports multiple processors.**
- **High processing capability.**
- **Suitable for complex applications.**

- Can interface with coprocessors.

11.Explain the Instruction Execution Cycle of 8086

The Instruction Execution Cycle is the sequence of operations performed by the 8086 microprocessor to execute an instruction. It consists of three basic stages:

1. Fetch
2. Decode
3. Execute

The Bus Interface Unit (BIU) performs the fetch operation, while the Execution Unit (EU) performs the decode and execute operations.

Steps in the Instruction Execution Cycle

Step 1: Fetch

- The Instruction Pointer (IP) contains the offset address of the next instruction.
 - The Code Segment (CS) provides the base address.
 - The BIU generates the 20-bit physical address.
 - The BIU fetches the instruction from memory.
 - The fetched instruction is stored in the 6-byte instruction queue.
-

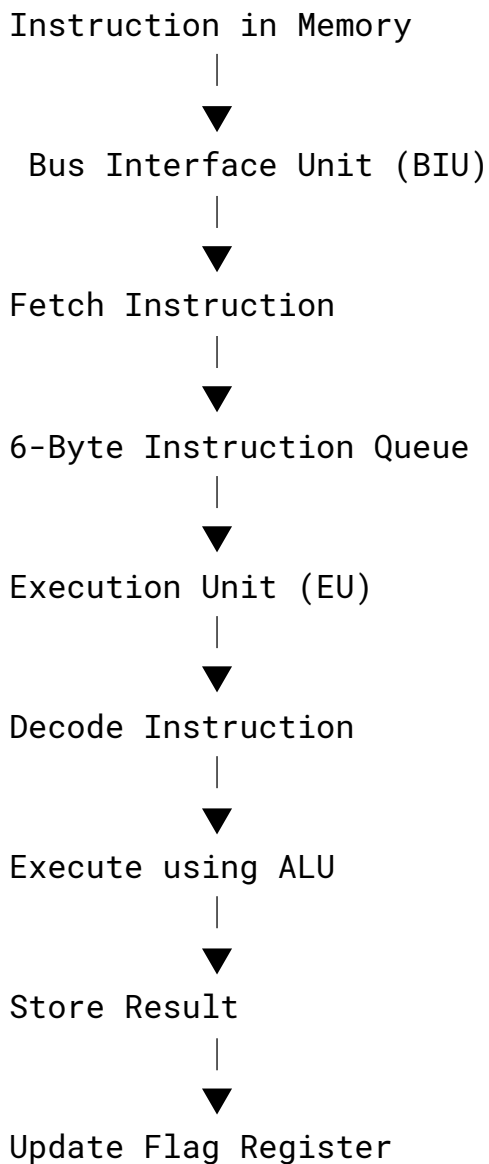
Step 2: Decode

- The Execution Unit (EU) takes the instruction from the queue.
 - The Control Unit decodes the instruction.
 - The processor identifies:
 - Operation to be performed
 - Source operand
 - Destination operand
 - Addressing mode
-

Step 3: Execute

- The ALU performs the required arithmetic or logical operation.
 - If memory access is needed, the EU requests the BIU.
 - The BIU transfers data between memory and the processor.
 - The result is stored in a register or memory.
 - The Flag Register is updated according to the result.
-

Flow Diagram



Role of BIU

The Bus Interface Unit performs the following functions:

- Generates the 20-bit physical address.
 - Fetches instructions from memory.
 - Stores instructions in the 6-byte queue.
 - Transfers data between memory and I/O devices.
 - Supports pipelining by fetching the next instruction while the current one is executed.
-

Role of EU

The Execution Unit performs the following functions:

- Takes instructions from the queue.
- Decodes the instructions.
- Executes arithmetic and logical operations.
- Updates registers and the Flag Register.

- Requests memory access through the BIU whenever required.
-

Pipelining During Execution

While the EU executes the current instruction:

- The BIU fetches the next instruction.
- The fetched instruction is placed in the 6-byte instruction queue.
- This overlapping of fetching and execution is called Pipelining.

Pipelining improves the execution speed of the 8086 microprocessor.

Advantages of the Instruction Execution Cycle

- Faster instruction execution.
- Efficient use of processor resources.
- Supports pipelining.
- Reduces processor idle time.
- Improves overall system performance.

12.(1) Write an 8086 Assembly Language Program to Add Two 16-bit Numbers

Aim

To write an 8086 assembly language program to add two 16-bit numbers.

Algorithm

1. Start the program.
 2. Initialize the data segment.
 3. Load the first 16-bit number into AX.
 4. Add the second 16-bit number to AX.
 5. Store the result.
 6. Terminate the program.
-

Program

DATA SEGMENT

NUM1 DW 2500H

NUM2 DW 1200H

RESULT DW ?

DATA ENDS

CODE SEGMENT

ASSUME CS:CODE, DS:DATA

START:

MOV AX, DATA

MOV DS, AX

MOV AX, NUM1

ADD AX, NUM2

MOV RESULT, AX

MOV AH, 4CH

INT 21H

CODE ENDS

END START

Explanation

Instruction	Explanation
MOV AX,DATA	Load data segment address
MOV DS,AX	Initialize DS register
MOV AX,NUM1	Load first number
ADD AX,NUM2	Add second number
MOV RESULT,AX	Store result
MOV AH,4CH	DOS terminate function
INT 21H	Return control to OS

Output

Result stored in RESULT.

(2)Write an 8086 Program to Multiply Two Numbers

Aim

To multiply two 8-bit numbers.

Algorithm

1. Initialize data segment.
2. Load first number into AL.
3. Multiply with second number.
4. Store result.
5. Stop execution.

Program

DATA SEGMENT

NUM1 DB 05H

NUM2 DB 04H

RESULT DW ?

DATA ENDS

CODE SEGMENT

ASSUME CS:CODE,DS:DATA

START:

MOV AX,DATA

MOV DS,AX

MOV AL,NUM1

MOV BL,NUM2

MUL BL

MOV RESULT,AX

MOV AH,4CH

INT 21H

CODE ENDS

END START

Explanation

Instruction	Explanation
MOV AL,NUM1	Load first number
MOV BL,NUM2	Load second number
MUL BL	Multiply AL by BL
MOV RESULT,AX	Store product
INT 21H	Exit program

Output

05 × 04 = 14H

Stored in RESULT.

(3)Write an 8086 Program to Perform 8-bit Subtraction

Aim

To subtract two 8-bit numbers.

Algorithm

1. Initialize DS.
2. Load first number.
3. Subtract second number.
4. Store result.
5. Stop.

Program

DATA SEGMENT

NUM1 DB 25H

NUM2 DB 10H

RESULT DB ?

DATA ENDS

CODE SEGMENT

ASSUME CS:CODE,DS:DATA

START:

MOV AX,DATA

MOV DS,AX

MOV AL,NUM1

SUB AL,NUM2

MOV RESULT,AL

MOV AH,4CH

INT 21H

CODE ENDS

END START

Output

25H - 10H = 15H

(4)Write an 8086 Program to Find the Largest Number in an Array

Aim

To find the largest number in an array.

Algorithm

1. Initialize DS.

2. **Load first array element into AL.**
 3. **Compare with remaining elements.**
 4. **Replace AL whenever a larger number is found.**
 5. **Store largest value.**
-

Program

DATA SEGMENT

ARRAY DB 25H,34H,18H,50H,29H

COUNT DB 05H

LARGEST DB ?

DATA ENDS

CODE SEGMENT

ASSUME CS:CODE,DS:DATA

START:

MOV AX,DATA

MOV DS,AX

LEA SI,ARRAY

MOV CL,COUNT

DEC CL

MOV AL,[SI]

NEXT:

INC SI

CMP AL,[SI]

JNC SKIP

MOV AL,[SI]

SKIP:

LOOP NEXT

MOV LARGEST,AL

MOV AH,4CH

INT 21H

Output

Largest Number =

50H

(5)Expression Evaluation

Evaluate: $(W+X+Y)-(U/V)$

Instead, it asks for the expression using:

- **Three-address instructions**
- **Two-address instructions**
- **One-address instructions**
- **Zero-address instructions**

Three-address

```
T1 = W + X
T2 = T1 + Y
T3 = U / V
T4 = T2 - T3
```

Two-address

```
MOV R1,W
ADD R1,X
ADD R1,Y
MOV R2,U
DIV R2,V
SUB R1,R2
```

One-address

```
LOAD W
ADD X
ADD Y
STORE TEMP

LOAD U
DIV V

SUB TEMP
STORE RESULT
```

Zero-address (Stack)

```
PUSH W
PUSH X
ADD

PUSH Y
ADD

PUSH U
PUSH V
DIV

SUB
POP RESULT
```

13.Explain CALL and RET Instructions in 8086

A subroutine is a group of instructions that performs a specific task and can be called from different parts of a program.

The CALL instruction transfers control from the main program to a subroutine, whereas the RET (Return) instruction transfers control back from the subroutine to the main program.

CALL Instruction

The CALL instruction is used to call a subroutine.

Before transferring control to the subroutine, the processor automatically stores the return address on the stack.

Syntax:

CALL LABEL

Working of CALL

1. The address of the next instruction is saved on the stack.
 2. Control is transferred to the specified subroutine.
 3. The subroutine starts execution.
-

Example

MAIN :

MOV AX,1000H

CALL ADDITION

MOV BX,AX

HLT

ADDITION :

ADD AX,2000H

RET

Explanation

- CALL ADDITION transfers control to the subroutine.
 - Before the jump, the return address is pushed onto the stack.
 - The subroutine executes.
 - RET returns execution to the instruction immediately after CALL.
-

RET Instruction

The RET instruction returns the program control from the subroutine to the calling program.

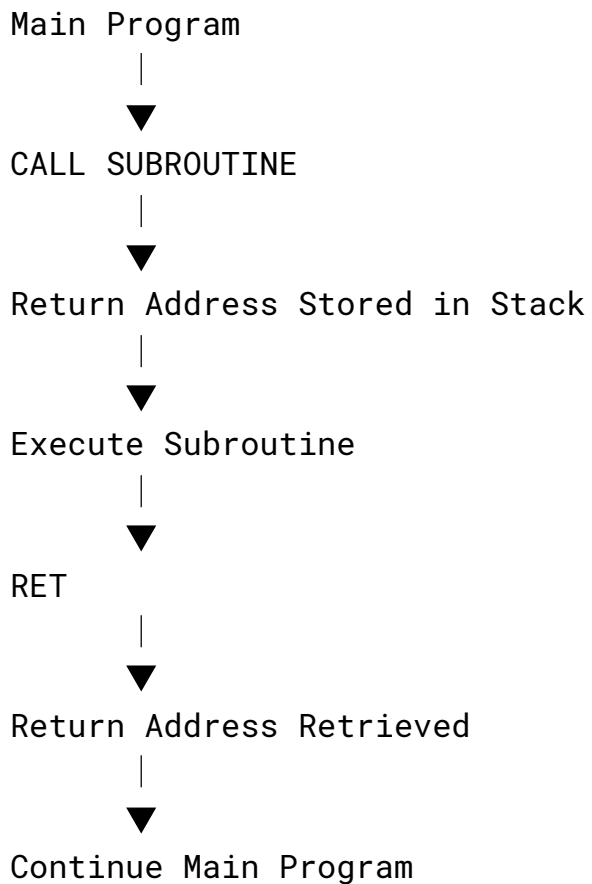
Syntax:

RET

Working of RET

1. The return address is popped from the stack.
 2. The address is loaded into the Instruction Pointer (IP).
 3. Execution resumes in the main program.
-

Flow of CALL and RET



Advantages of CALL and RET

- Avoid code repetition.
 - Reduce program size.
 - Improve program readability.
 - Simplify debugging.
 - Support modular programming.
-

Difference Between CALL and RET

CALL	RET
Transfers control to a subroutine	Returns control to the calling program
Pushes return address onto the stack	Pops return address from the stack
Starts execution of a subroutine	Ends execution of a subroutine
Uses stack to save return address	Uses stack to restore return address

14. Difference Between NEAR Jump and FAR Jump

A jump instruction transfers program control from one location to another.

There are two types of jumps in the 8086:

1. Near Jump
2. Far Jump

Near Jump

A Near Jump transfers control within the same code segment.

- Only the Instruction Pointer (IP) is updated.
- The Code Segment (CS) remains unchanged.

Example

```
JMP NEXT
```

Far Jump

A Far Jump transfers control to a different code segment.

- Both CS and IP are updated.
- Used when the destination is outside the current code segment.

Example

```
JMP 5000H:2000H
```

Difference Between Near Jump and Far Jump

Near Jump	Far Jump
Transfers control within the same code segment	Transfers control to another code segment
Only IP changes	Both CS and IP change
Faster execution	Comparatively slower
Smaller instruction size	Larger instruction size
Used for short-distance jumps	Used for long-distance jumps

Advantages

Near Jump

- Faster execution.
- Less memory required.
- Simple implementation.

Far Jump

- Accesses code in different segments.
- Supports large programs.
- Provides flexibility in program organization.



SAQs

1. Define Microprocessor.

A microprocessor is a programmable electronic device fabricated on a single silicon chip. It consists of an Arithmetic Logic Unit (ALU), Control Unit, and Registers. It accepts binary data as input, processes it according to the instructions stored in memory, and produces the output.

2. Differentiate between Microprocessor and Microcontroller.

Microprocessor	Microcontroller
Contains only CPU.	Contains CPU, RAM, ROM, and I/O on a single chip.
Requires external memory and I/O devices.	Has built-in memory and I/O ports.
Used for general-purpose computing.	Used for embedded systems.
More power consumption.	Less power consumption.

3.What is an Instruction Queue?

An Instruction Queue is a 6-byte FIFO memory present in the Bus Interface Unit (BIU) of the 8086 microprocessor. It stores prefetched instructions before execution and supports pipelining, thereby improving processor speed.

4.What is Pipelining?

Pipelining is the technique in which the Bus Interface Unit (BIU) fetches the next instruction while the Execution Unit (EU) executes the current instruction. It enables simultaneous fetching and execution, thereby increasing processor performance.

5.What is Memory Segmentation?

Memory Segmentation is the process of dividing the available memory into smaller sections called segments. In the 8086 microprocessor, the 1 MB memory is divided into four 64 KB segments to improve execution and memory access speed.

6.List the Segment Registers.

The 8086 microprocessor contains four 16-bit segment registers:

- Code Segment (CS)
- Data Segment (DS)
- Stack Segment (SS)
- Extra Segment (ES)

These registers store the base addresses of their respective memory segments.

7.What is the Function of BIU?

The Bus Interface Unit (BIU) interfaces the 8086 with memory and I/O devices. Its functions are:

- Generates the 20-bit physical address.
 - Fetches instructions from memory.
 - Transfers data between memory and I/O.
 - Maintains the 6-byte instruction queue.
-

8.What is the Function of EU?

The Execution Unit (EU) decodes and executes instructions received from the instruction queue. It performs arithmetic and logical operations using the ALU and controls program execution.

9.What is the Size of the Address Bus in 8086?

The 8086 microprocessor has a 20-bit address bus, enabling it to access 2^{20} bytes (1 MB) of memory.

10.What is the Size of the Data Bus in 8086?

The 8086 microprocessor has a 16-bit data bus, allowing it to transfer 16 bits of data in one operation.

11.Define Addressing Mode.

An addressing mode is the method by which an instruction specifies the location of its operand. The operand may be located in a register, memory, or within the instruction itself.

12.What is Immediate Addressing Mode?

In Immediate Addressing Mode, the operand is specified directly in the instruction itself.

Example:

```
MOV AL, 35H
```

13.What is Register Indirect Addressing Mode?

In Register Indirect Addressing Mode, the offset address of the operand is stored in one of the registers BX, BP, SI, or DI.

Example:

MOV AX, [BX]

14.What is the Purpose of the Flag Register?

The Flag Register is a 16-bit register that stores the status of arithmetic and logical operations and controls processor execution. It contains status flags and control flags.

15.What are String Instructions?

String Instructions are instructions used to process strings or arrays of data. They perform operations such as moving, comparing, scanning, loading, and storing string data.

Examples:

- MOVS
 - CMPS
 - SCAS
 - LODS
 - STOS
-

16.What is a Machine Cycle?

A Machine Cycle is the time required by the microprocessor to complete one basic operation, such as memory read, memory write, I/O read, or I/O write. It consists of one or more T-states.

17.What is a T-State?

A T-State (Timing State) is one clock period of the processor. It is the smallest unit of time required to perform an operation in a machine cycle.

18.What is an Opcode?

An Opcode (Operation Code) is the part of an instruction that specifies the operation to be performed by the processor, such as ADD, SUB, MOV, or MUL.

19.What is an Assembler?

An Assembler is a software program that converts assembly language programs into machine language (binary code) so that they can be executed by the microprocessor.

20.What is the Purpose of the LOOP Instruction?

The LOOP instruction is used to execute a group of instructions repeatedly. It automatically decrements the CX register by 1 and continues looping until CX becomes zero.